תרגיל מעשי: הוספת חיפוש מהיר עם Redis לחנות

Docker & Redis - תרגיל מתקדם (.NET Core 8)

רקע למשימה

בחנות האינטרנט שלכם, לקוחים רוצים לחפש מוצרים במהירות. כרגע, כל חיפוש עובר ישירות למסד הנתונים, מה שיוצר עומס ואיטיות כשיש הרבה משתמשים.

המשימה שלכם: להוסיף Redis כשכבת cache לחיפושים מהירים!

מה זה Redis?

Redis (Remote Dictionary Server) הוא מסד נתונים בזיכרון (in-memory) שמאפשר:

- חיפושים מהירים במיוחד
- שמירת מידע זמני (cache)
- ספירת צפיות/קליקים
- תפוגה אוטומטית של מידע (TTL - Time To Live)

למה Redis?

- חיפוש ב-Redis: ~0.1ms
- חיפוש ב-SQL Server / PostgreSQL: ~10-50ms
- **פי 100-500 יותר מהיר!**

דרישות המשימה

חלק א': הגדרת הסביבה (20 נקודות)

1. הוסיפו Redis ל-docker-compose.yml

- השתמשו ב-image: redis:7-alpine
- הגדירו port: 6379:6379
- הוסיפו volume לשמירת מידע: redis_data:/data

2. עדכנו את ה-API service

- הוסיפו משתנה סביבה: Redis__ConnectionString=redis:6379
- הוסיפו depends_on: redis

3. התקינו את ספריית Redis ב-.NET

- הריצו: dotnet add package StackExchange.Redis

4. הגדירו Redis ב-Program.cs

- הוסיפו Redis ל-Dependency Injection עם AddStackExchangeRedisCache
- קראו את ה-connection string מ-configuration

5. ודאו שהכל עובד

- הריצו: docker-compose up --build
- ודאו שרואים: api, db, redis

חלק ב': מימוש חיפוש עם Cache (50 נקודות)

צרו endpoint: GET /api/products/search?query=laptop

הלוגיקה הנדרשת:

1. צרו Service לניהול Cache

- צרו IProductCacheService interface
- מימשו ProductCacheService שמשתמש ב-IDistributedCache

2. פעולות ה-Service:

- GetSearchResultsAsync(string query) - קריאה מ-cache
- SetSearchResultsAsync(string query, List<Product> products) - שמירה ב-cache
- ClearSearchCacheAsync() - ניקוי cache

3. ב-Controller:

- בדקו אם יש ב-cache
- אם כן - החזירו (cache HIT)
- אם לא - שאלו את ה-DB, שמרו ב-cache, החזירו (cache MISS)

4. תגובה ללקוח:

- "source: "cache" או "database"
- data: רשימת המוצרים

טיפים:

- השתמשו במפתח ייחודי לכל חיפוש (למשל: search:{query})
- Redis שומר רק strings - השתמשו ב-JsonSerializer
- הגדירו TTL של 5 דקות עם DistributedCacheEntryOptions

חלק ג': ניקוי Cache (15 נקודות)

בעיה: אם מוצר מתעדכן, ה-cache עדיין מחזיק מידע ישן!

הפתרון:

כשמוצר מתווסף/מתעדכן/נמחק (POST/PUT/DELETE /api/products), קראו ל-ClearSearchCacheAsync()

איך לממש:

- במחלקת ה-Service, צרו method שמוחק את כל מפתחות החיפוש
- תצטרכו להשתמש ב-IConnectionMultiplexer ישירות לפעולה זו
- קראו ל-method הזה בכל פעולת עדכון

חלק ד': מוצרים פופולריים - BONUS (15 נקודות)

משימה: שמרו איזה מוצרים הכי נצפים

איך:

בכל בקשה ל-GET /api/products/{id}, הגדילו מונה למוצר זה
צרו endpoint: GET /api/products/top שמחזיר את 10 המוצרים הנצפים ביותר

טיפ: Redis תומך ב-Sorted Sets - השתמשו ב-IConnectionMultiplexer לפעולות אלו

בדיקות - איך תדעו שזה עובד?

1. בדיקת חיפוש בסיסי

- חיפוש ראשון (צריך להיות מה-DB) - תשובה: source: database
- חיפוש שני (צריך להיות מ-cache) - תשובה: source: cache

2. בדיקה ידנית ב-Redis

- היכנסו ל-redis-cli: Redis: docker exec -it <redis-container-name>
- בדקו מה שמור: <KEYS *, GET <key-name>, TTL <key-name

3. בדיקה עם Swagger

- גשו ל-http://localhost:5000/swagger
- בדקו את endpoint החיפוש
- שימו לב לשדה source בתגובה

שגיאות נפוצות - איך לפתור

"Unable to connect to Redis"

- Redis רץ? docker-compose ps
- ה-connection string נכון? צריך להיות redis:6379 (לא localhost!)
- יש depends_on: redis ב-docker-compose?

"Serialization error"

- ה-Models שלכם serializable? (אין circular references?)
- השתמשתם ב-System.Text.Json.JsonSerializer?
- בדקו null values

"Cache לא מתרוקן"

- אתם באמת מוחקים מפתחות כשמעדכנים?
- המפתחות שאתם מוחקים הם הנכונים?
- בדקו את ה-pattern matching

קריטריוני הגשה

חובה (85 נקודות)

- Redis רץ ב-docker-compose (20)
- חיפוש מוצרים עובד עם cache (30)
- TTL מוגדר (3-10 דקות) (10)
- הצגת source במענה (10)
- ניקוי cache בעדכונים (15)

בונוס (15 נקודות)

- מוצרים פופולריים (Top 10)

דרישות נוספות

- README עם הסבר איך להריץ

- צילומי מסך / GIF של הפעולה
- הערות בקוד (XML comments)

משאבים לעזרה

- תיעוד Redis: https://redis.io/commands
- StackExchange.Redis: https://github.com/StackExchange/StackExchange.Redis
- .NET Redis Caching: https://learn.microsoft.com/en-us/aspnet/core/performance/caching/distributed
- Docker Redis Image: https://hub.docker.com/_/redis

טיפים להצלחה

עבדו בשלבים - תחילה רק חיפוש בסיסי, אחר כך שאר הפיצ'רים

השתמשו ב-ILogger - תוכלו לראות בדיוק מה קורה (cache HIT/MISS)

התחילו עם TTL קצר - 30 שניות לבדיקות, אחר כך העלו ל-5 דקות

תבדקו בעצמכם - השתמשו ב-Swagger, Postman, או curl

שאלו שאלות - אם משהו לא ברור!

בהצלחה!